



Build Automation

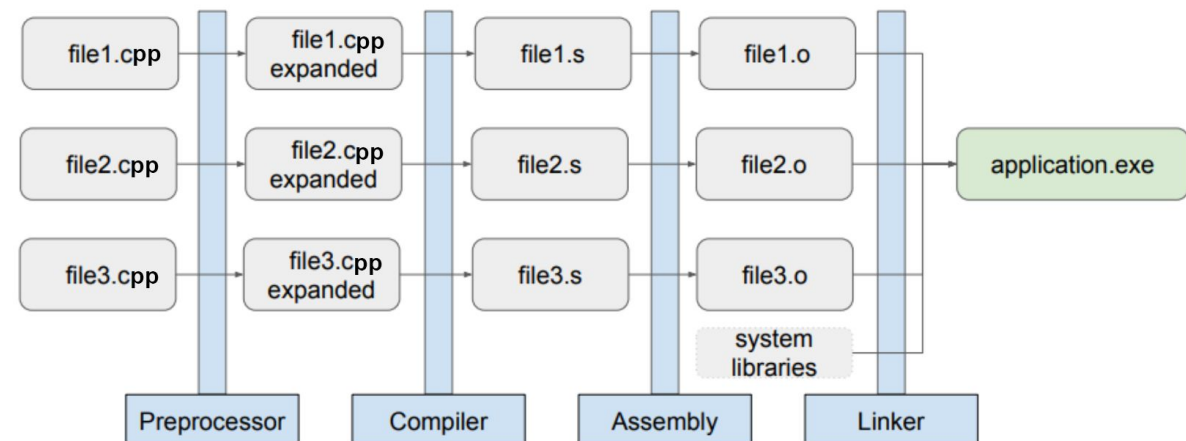
GCC, Make & Makefiles and CMake

Compilation Using GCC

❖ Build process of a C++ program

- **Preprocessor:** '#'-prefixed lines for includes, replacing macros, conditional compilation, etc.
- **Compiler:** Generate assembly code from the preprocessed code, checks the code for errors
- **Assembler:** Makes machine instructions (object file) from the generated assembly code
- **Linker:** Resolves symbols (function calls, global variables...) between software components and system libraries

- ❖ `g++ -E file.cpp -o file.ii => Preprocessed code`
- ❖ `g++ -S file.cpp -o file.s => Assembly code`
- ❖ `g++ -c file.cpp => Object file (file.o is generated)`
- ❖ `g++ file1.o file2.o file3.o -o app => Linked file(app)`
- ❖ `g++ main.cpp -save-temps -o main`
- ❖ `g++ file1.cpp file2.cpp file3.cpp -o application`



Compilation Using GCC

❖ GCC has so many flags and options to control compilation. [🔗 The GCC Manual](#)

❖ Some useful options in the Preprocessor phase

➤ `g++ -E main.cpp -o main.ii`

- Saves the preprocessed code in a `.ii` file

➤ `g++ -E -C main.cpp -o main.ii`

- Saves the preprocessed code in a `.ii` file
- Prevents the preprocessor from removing comments

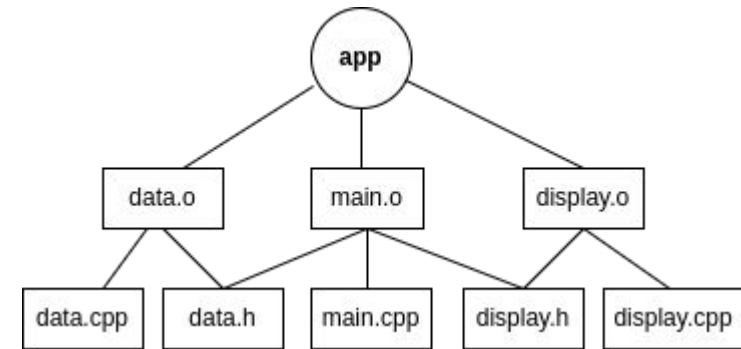
➤ `-D name[=definition]` to define a macro

➤ `-U name` to undefine a macro defined on the command

line or a G++ predefined macro. The `-D` and `-U` options

are processed in the order in which they occur on the command line

➤ `-I directory` is used to specify the directory path of header files.



A Simple Program

- `g++ -c main.cpp`
- `g++ -c data.cpp`
- `g++ -c display.cpp`
- `g++ data.o main.o display.o -o app`

Compilation Using GCC

- ❖ `g++ -S -fverbose-asm main.cpp` generates `main.s` and includes variable names as comments in the assembly file
- ❖ Some useful options in the Compilation phase
 - `g++ -c main.cpp` generates `main.o` as the object file and
 - `g++ -c -g main.cpp` generates `main.o` for debugging
- ❖ `g++ -fsyntax-only main.cpp` only checks the syntax of `main.cpp`
- ❖ It is possible to compile multiple files in one command
 - `g++ -c file1.cpp file2.cpp`; it is equivalent to `g++ -c file1.cpp` and `g++ -c file2.cpp`
 - But in a large program compilation of all files in one command can cause problems
 - Larger memory is required and the compilation takes more time

Compilation Using GCC

❖ Some useful options in the Linking phase

- The standard library is in **libstdc++.a** (.a stands for **archive**) or in **libstdc++.so** (so stands for **shared object**) in the default lib path. To link a non-standard library, **-llibrary** for liblibrary.a shall be used
- If the target supports shared library, liblibrary.so will be linked if **-static** option is not used
- If the library is not in the default lib path, **-Ldirectory** shall be used to specify the library path
 - E.g. g++ -o test module.cpp test.cpp -lunity

❖ If the target supports shared libraries, a shared library

- Is a special object that can be linked to a program at runtime and has some advantages:
 - A program's executable file is smaller
 - Shared modules permit modular updating
 - More efficient use of the available memory.
- g++ **-shared** -o lib<library-name>.so library-name.o creates a shared library
 - To compile the code use **-fPIC** to generate position-independent code

Some Useful Tools

- ❖ An executable linked with a **.so** file contains some relocation and symbol table information that allow references to code and data in the **.so** to be resolved at run time.
- ❖ To create a static library(an archive) of object files you can use **ar**.
 - E.g. **ar -crv lib<name>.a file1.o file2.o file3.o ...**
- ❖ To list symbols in an object file or a static library you can use **nm**.
- ❖ To display all the information in an object binary file you can use **objdump**.
- ❖ To list all the printable strings in a binary file you can use **strings**.
- ❖ Unlike C, the C++ compiler performs name mangling.
 - You know that for C linkage codes we shall use **extern "C"** to skip name mangling.
 - To demangle a symbol to into a user-level name you can use **c++filt**.
- ❖ To list all shared libraries which a program depends on, you can use **ldd**.

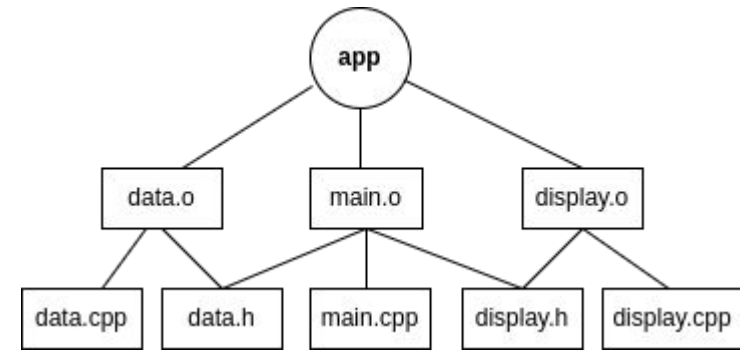
Compilation Using GCC

- ❖ G++ supports different C++ versions using **-std=xxx**; e.g. `g++ -std=c++14 main.cpp`
- ❖ **-pedantic-errors** used to fail on non-standard usage.
- ❖ **-pg** adds profiling information to a program for analysis
 - **gprof** generates a call graph and execution time of functions
 - E.g. `g++ -pg main.cpp -o main && ./main && gprof ./main`
- ❖ **-Wall** generates a wide range of warnings
- ❖ **-Werror** is used to fail on all warnings
- ❖ **-Wextra** generates warnings on legal but questionable usage.
 - E.g. check if an unsigned variable is lesser than zero
- ❖ **-Wconversion** generates warnings on implicit type conversions that change values

Make and Makefiles

❖ Compilation of a small program

- `g++ -c main.cpp` (creates `main.o`)
- `g++ -c data.cpp` (creates `data.o`)
- `g++ -c display.cpp` (creates `display.o`)
- `g++ data.o display.o main.o -o app`



❖ It is clear we can not compile a large and complex program manually over and over

❖ **make** automates the process of building programs of any size and complexity

- It manages the individual rules that define how to build various targets
- It does not build unchanged files in projects by comparing files timestamps
- It can build different parts of a large program and multiple execution units in one build
- It uses special rules and syntax to build targets according their dependencies

Make and Makefiles

❖ The problem

- To make an executable program we need to link compiled object files
- To generate object files, we need to compile C++ source files
- The source files need to be preprocessed to include their header files.
- If we edit a source or header file, then any file that was directly or indirectly generated from it needs to be rebuilt.

❖ The solution

- **make** organizes the above tasks in the form of rules.
 - **make** has a special syntax for its rules.
- ***make -f filename*** tells make which file contains the rules
- Usually this option is omitted and **make** looks for a file with the default name **makefile** or **Makefile**.

Makefiles

❖ General form of rules in makefiles

➤ **targets: dependencies**

<TAB>**command**

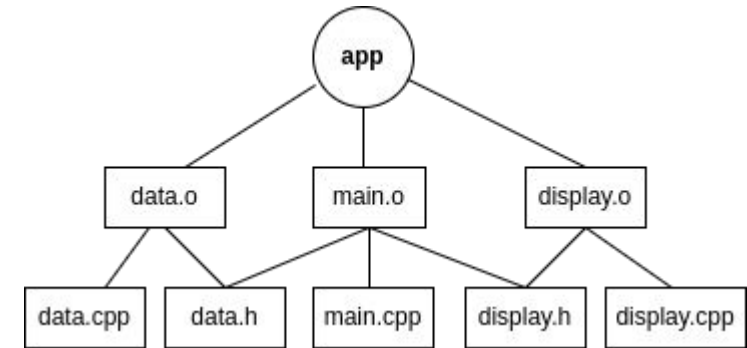
<TAB>**command**

<TAB>...

- Targets can be one or more
 - The first target must be placed at the beginning of the line, with no whitespace to its left
- Dependencies can be zero or more
- Each command line must start with a tab

❖ In addition to rules, makefiles also contain

- Comments, variable assignments, macro definitions, include and conditional directives, loop and functions.



A basic Makefile

```
app: data.o display.o main.o
    g++ -o app data.o display.o main.o

data.o: data.cpp data.h
    g++ -c data.cpp

display.o: display.cpp display.h
    g++ -c display.cpp

main.o: main.cpp data.h display.h
    g++ -c main.cpp
```

Makefiles

- ❖ **make** parses a Makefile and builds a dependency tree for targets
- ❖ Then it uses the dependency tree to build the desired targets
 - A Makefile is not executed in sequential order
 - If a target is older than any dependencies (directly or indirectly)
 - **make** executes the target commands.
 - A rule with **no** dependency tells only **how** to build the target, not **when** to build it
- ❖ Phony Targets: A target which is not a file
 - Just it is name of a recipe to be executed
 - It is used to avoid a conflict with a file of the same name
 - Phony targets are defined as .PHONY
 - E.g. clean, install, all and etc. To make: **make clean**

A basic Makefile

.PHONY: clean mkbuild

clean:

rm -rf build

mkbuild:

mkdir -p build

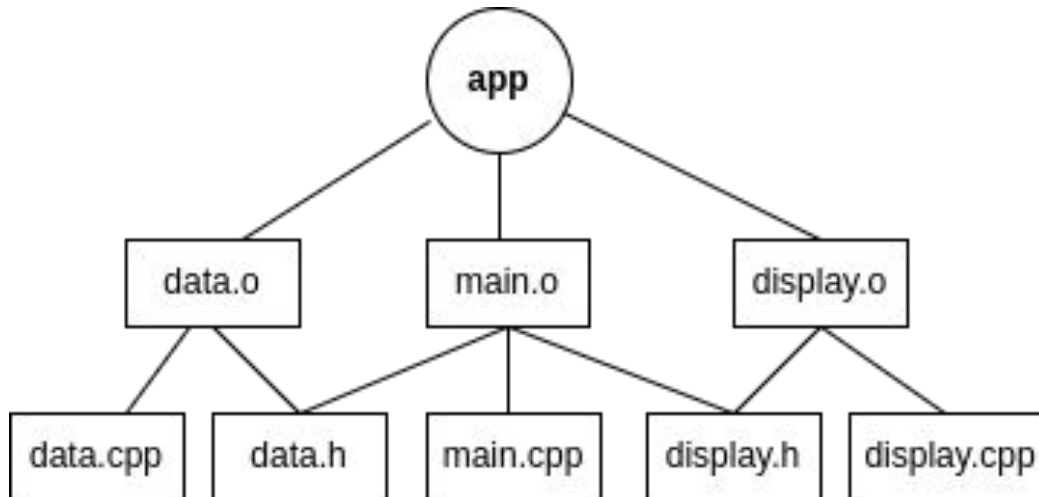
Makefiles

❖ To build a target run ***make target***. E.g. ***make clean***

➤ ***make*** by default runs the top target.

■ To set the default target: E.g. **`.DEFAULT_GOAL := all`**

➤ ***all*** is the top-level target the makefile knows about.



A basic Makefile

all: app

app: data.o display.o main.o
g++ -o app data.o display.o main.o

data.o: data.cpp data.h
g++ -c data.cpp

display.o: display.cpp display.h
g++ -c display.cpp

main.o: main.cpp data.h display.h
g++ -c main.cpp

clean:
rm -f *.o

.PHONY: all clean

Makefiles

- ❖ **make** runs the commands in a rule individually in a separate shell instance
- ❖ Make sure that no command depends on the side effects of the previous command
- ❖ In the first rule, **rm** is not run in the **build** folder.
- ❖ To run several commands in the same shell
 - Put the commands in one line separated by semicolon
 - If you want to have more than one line use backslashes
 - Running multiple commands in the same shell can speed up processing
- ❖ There are useful command line [options](#)
 - **-B**: Consider all targets out-of-date.
 - **-j [jobs]**: Specifies the number of recipes (jobs) to run simultaneously.
 - **-n**: Print the recipe that would be executed, but do not execute it
 - **-s**: Silent operation; do not print the recipes as they are executed

```
clean:  
  cd build  
  rm *.o
```

```
clean:  
  cd build; \  
  rm *.o
```

Makefiles

- ❖ Using **#** you can comment
 - To comment between lines of a rule, **#** shall be in the beginning of the line
 - For **multiline** comments you can use **backslash** (\)
- ❖ In makefiles we can use variables. Type of all variables is string
- ❖ Variable names can contain any character except **:**, **=** and **#**
 - But to be compatible with shell, you should use only **letters**, **digits** and **underlines**
 - To use variables they shall be enclosed in parentheses and prefixed with **\$**.
 - E.g. **CC = g++** and we can use CC like **\$(CC)**
- ❖ When make applies a rule, it evaluates the variables used in the rule
- ❖ make evaluates variables by expanding them in two ways
 - ***recursively expanded*** or ***simply expanded***

Makefiles

❖ In recursively expanded variables

- The reference of variables in the right side of assignments are stored, not their values.
- = is used as the assignment operator
- In the example y is recursive and the reference of \$(x) is not expanded until y is evaluated.

```
x = hello
y = $(x)
# Both $(x) and $(y) will now yield "hello"
x = world
# Both $(x) and $(y) will now yield "world"
```

❖ In simply expanded variables

- Variable references in the value are expanded immediately on assignment
- The expanded values are stored, not references
- := s used as the assignment operator

```
x := hello
y := $(x)
# Both $(x) and $(y) will now yield "hello"
x := world
# $(x) will now yield "world", and $(y) will yield "hello"
```

❖ += operator is used to append more characters

to the value of a recursively expanded variable. E.g. y = Hello and y += World!

Makefiles

- ❖ Variable ending substitution - to substitute ending substrings

`$(variable_name:ending=new_ending)`

`SOURCES = main.cpp data.cpp display.cpp`

`OBJECTS = $(SOURCES:.cpp=.o)`

- ❖ It is possible to create and assign variables in the command line.

- **make** [options] [variable_assignments] [targets separated by spaces]
- E.g. **make number=123 clean run**. Now number can be used in the Makefile

- ❖ Predefined variables in Makefiles

- `$@` is name of the file being created by the rule.
- `$*` is the prefix that is common to both target and dependent file
- `$<` is the first dependency
- `^` is list of the dependencies.
- `+` is list of the dependencies, excluding duplicates.
- `?` is list of the dependencies that are newer than the target.

`display.o: display.cpp display.h`

`$(CC) $(CFLAGS) -c $< -o $@`

Makefiles

❖ Pattern Rules

- Generalizing the rules to avoid explicit rules for each file
- An pattern rule is where no specified target is given, but instead a wildcard (%)

```
%.o : %.cpp
$(CC) $(CFLAGS) $< -o $@
```

❖ We can have **macros** in makefiles. Macros are defined using **define** and **endif**

- There is no difference between macros and variables
- A macro is used like a variable. E.g. \$(CLEAN)

```
define CLEAN
@echo Clean The Project
rm -rf build
endif
```

❖ **make** supports user-defined functions and has some built-in functions

❖ General form of calling a function is

\$(function_name arguments_list_separated_by_comma)

❖ **make** provides some useful [functions](#)

❖ User defined functions are like function-like macros

Makefiles

- ❖ A **user defined function** can be defined using a **macro** or a **variable**
 - Functions in **make** are variables with parameter references (\$1, \$2, \$3, etc.)
- ❖ A user defined function can be called using the built-in **call** function.
 - The syntax of the call function is: **\$(call variable,param,param,...)**
 - In the example **foo** contains **b a**

```
reverse = $(2) $(1)
foo = $(call reverse,a,b)
```
- ❖ Some useful built-in functions
 - **\$(subst find_text,replacement_text,original_text)**
 - Is used to find and replace a **find_text** with **replacement_text** in **original_text**
 - **\$(wildcard pattern)**
 - Expands to a list of existing filenames that match the pattern. E.g. **\$(wildcard *.c)**
 - **\$(patsubst find_pattern,replacement_pattern,original_text)**
 - Exchanging part of filename. E.g. **objects := \$(patsubst %.c,%.o,\$(wildcard *.c))**

Makefiles

- ❖ **make** supports directives. Look at the [directives](#)
 - **define** and **endef** to define macros
 - Conditional directives: **ifeq**, **ifneq**, **ifdef**, **ifndef**, **else**, **endif**
 - **include** is used to include a makefile in another one
 - **g++ -M**, **-MM** and **-MMD** generate the dependencies automatically
 - Similar to include in C++ but works differently
 - We can include multiple files in one include
 - E.g. **include** defaults.mk \$(SRCDIR)/file.mk foo bar
 - Possible to use wildcards like * and ? in the file names
 - E.g. **include** defaults \$(SRCDIR)/*.mk
 - **override** is used to change variables defined in command line. E.g. **make VAR=10 run** and in the Makefile we can have: **override VAR = 20**

```
OBJ = test.o fizzbuzz.o

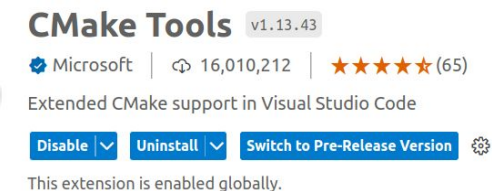
ifdef SHAREDLIB
    LIB += unity.so
else
    OBJ += unity.o
endif

test: $(OBJ) $(LIB)
    $(CC) -o $@ $^
%.so : %.o
    $(CC) -shared -o $@ $<
```

```
TEXT = aaaa bbbb cccc dddd

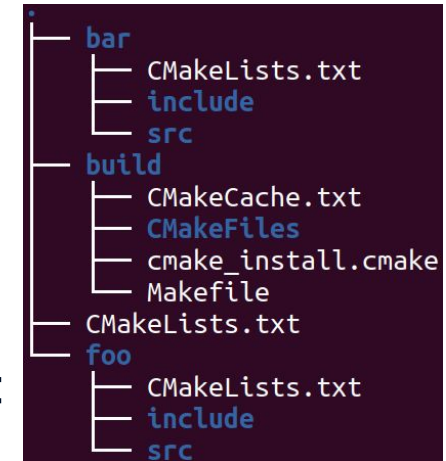
all:
    $(foreach v, $(TEXT), echo $(v);)
```

- ❖ CMake is an open-source and cross-platform tool family to build, test and package.
- ❖ CMake is the most used tool to control software compilation process using simple platform and compiler independent configuration files and [generate Makefiles and other build automation tools](#) like [ninja](#).
- ❖ CMake has its own syntax to define **build configuration** file(s) for a project.
 - A build configuration file is a text file exactly named **CMakeLists.txt**
 - In Visual Studio Code install the **CMake Tools** extension. It provides syntax highlighting, auto-completion and configuration.
- ❖ In order to use cmake to build a project we shall provide CMakeLists.txt file(s) to cmake and then cmake generates files including Makefile(s) to build the project.



- ❖ Usually a project contains a **build** directory in which **cmake** is called.
- ❖ It is a common practice to create **build** in the root directory of the project if it does not already exist.
- ❖ **cmake** can be used to **build** at different levels.
- ❖ **cmake** [provides interface](#) to perform all the build process. For example:
 - **cmake -S <source-dir> -B <build-dir>** to generate a project build system
 - **cmake --build <dir>** to build a project.
 - **cmake --install <dir>** to install a project
 - **cmake -E <command>** to run a command line tool
 - **cmake --help[-<topic>]** to display help

```
mehri@ubuntu:~/project$ ls
CMakeLists.txt  main.cpp
mehri@ubuntu:~/project$ cmake -S . -B build && cmake --build build
```



```
M CMakeLists.txt
1  cmake_minimum_required(VERSION 3.5.1)
2  project(sample)
3
4  add_subdirectory(foo)
5  add_subdirectory(bar)
```

```
mehri@ubuntu:~/project$ mkdir build
mehri@ubuntu:~/project$ cd build
mehri@ubuntu:~/project/build$ cmake ..
mehri@ubuntu:~/project/build$ make
```

- ❖ Commands are separated by new lines. Comments start with **#**
- ❖ Every **CMakeLists.txt** starts with defining a minimum version of CMake to be used:

```
cmake_minimum_required(VERSION version_number)
```

```
cmake_minimum_required(VERSION min_version...max_version)
```

- ❖ Every **top-level CMakeLists.txt** file should define a project

- **project_name** is mandatory
- The other arguments are optional

```
project(project_name VERSION 1.0
        DESCRIPTION "CMake Example"
        HOMEPAGE_URL "www.project.url"
        LANGUAGES CXX)
```

- ❖ To add targets, use the following commands:

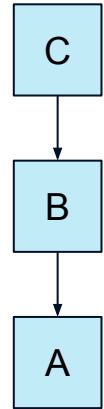
- **add_executable** to build an executable
- **add_library** to build a library **add_executable**(targetName [EXCLUDE_FROM_ALL] source1 [source2 ...])
- **add_custom_target** to add a custom target

```
add_library(targetName [OBJECT|STATIC | SHARED | MODULE] [EXCLUDE_FROM_ALL] source1 [source2 ...])
```

- ❖ CMake can build any number of targets as needed.
- ❖ The default target is ***all*** to build all targets, except those which have set the **EXCLUDE_FROM_ALL** property.
- ❖ Normal executables are added as targets with **globally** unique names within the project.
- ❖ Normal libraries could be defined as either **STATIC** for static libraries, **SHARED** for dynamic libraries, and **MODULE** for plug-ins. Modules are loaded dynamically at run-time rather than being linked directly to a library or executable
- ❖ Possible to set the **BUILD_SHARED_LIBS** variable to **YES** or **NO** using the **set** command before each target or globally. E.g. `set (BUILD_SHARED_LIBS YES)`
- ❖ To link libraries to a target, **target_link_libraries** should be used.

```
target_link_libraries(targetName  
  <PRIVATE|PUBLIC|INTERFACE> item1 [item2 ...]  
  ...  
)
```


- ❖ Assume that library B needs library A. It means that library B uses library A.
 - **PRIVATE** dependency means that A is only used in the **internal implementation** of B and if C uses B, it does not need to know about A.
 - **PUBLIC** dependency means that A is used in both the internal implementation and the interface of B. It means that C cannot use B without using A.
 - **INTERFACE** dependency means that A is used only in the interface of B and in order to use library B, parts of library A must also be used. E.g. a header-only library



- ❖ To add include directories to a target:
- ❖ To add preprocessor definitions for a target:

```
target_include_directories(<target> [SYSTEM] [AFTER|BEFORE]
                           <INTERFACE|PUBLIC|PRIVATE> [items1...]
                           [<INTERFACE|PUBLIC|PRIVATE> [items2...] ...])
```

```
target_compile_definitions(targetName
    <PRIVATE|PUBLIC|INTERFACE> item1 [item2 ...]
    [<PRIVATE|PUBLIC|INTERFACE> item3 [item4 ...]]
    ...
)
```

```
target_compile_definitions(MyTarget PRIVATE
    ADD_DEBUG_PRINTS=1
    ABCDE )
```

- ❖ To add preprocessor definitions to the compilation of source files, use

```
add_compile_definitions(<definition> ...)
```

E.g `add_compile_definitions(DEBUG NUM=12)`

- ❖ To add compile options for a target:

E.g `target_compile_options(MyTarget PRIVATE -Wall -g)`

```
target_compile_options(<target> [BEFORE]
    <INTERFACE|PUBLIC|PRIVATE> [items1...]
    [<INTERFACE|PUBLIC|PRIVATE> [items2...] ...])
```

- ❖ To add options to the compilation of source files

```
add_compile_options(<option> ...)
```

E.g `add_compile_options(-Wall -Wextra -pedantic -Werror)`

- ❖ To define and undefine variables, you can use `set` and `unset`.
- ❖ Variable names can contain letters, numbers and underscores, with letters being case-sensitive.
- ❖ Variables have scopes. **PARENT_SCOPE** is used to create/modified variables in subdirectories or functions with parent scope.

```
set(varName value... [PARENT_SCOPE])
```

E.g. `set(var "value")` and `unset(var)`

```
set(multiLine [[
First line
Second line]])
```

- ❖ To get access to value of a variable enclose it using `${}`. E.g. `${MY_VAR}`

`set(MY_VAR 1234 "Hello World")` creates a list

E.g `message("${MY_VAR}")`

- ❖ To retrieve and set environment variables:

E.g `set(ENV{PATH} "$ENV{PATH}:/opt/myDir")`

- ❖ To define cache variable: `set(varName value... CACHE type "docstring" [FORCE])`

➤ Unlike normal variables which have a lifetime limited to the processing of the CMakeLists.txt file, cache variables are stored in a special file named **CMakeCache.txt**

- ❖ There are a lot of provided variables by cmake.

- ❖ Examples:

```
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

```
set_target_properties(myTarget PROPERTIES
  CXX_STANDARD 11
  CXX_STANDARD_REQUIRED YES
  CXX_EXTENSIONS NO
)
```

- ❖ Possible to have **if statements**, **foreach** and **while** statements, **functions** and **macros**.
 - Look at [cmake commands](#)
- ❖ In a multiple directory project it is possible to have multiple CMakeLists.txt files and build in different levels by bringing subdirectories to the build. It means that the build logic can be distributed across the directory hierarchy rather than forcing everything to be defined at the top-most level
- ❖ To add a subdirectory into the build, use **add_subdirectory(subDir [binDir] [EXCLUDE_FROM_ALL])**
- ❖ The subdirectory must have its own CMakeLists.txt file which will be processed at the point where `add_subdirectory()` is called.
- ❖ [Modules](#) are pre-built CMake code built on top of the core language features
- ❖ Modules are collected together and provided in a single directory as part of a CMake release

- ❖ The `include` command is used to include a module or generally a CMakeLists.txt
 - `include(<file|module> [OPTIONAL] [RESULT_VARIABLE <var>] [NO_POLICY_SCOPE])`
 - The `include()` command to essentially inject the module code into the current scope.
 - A module file name is in the format of `<modulename>.cmake`
 - If **OPTIONAL** is present, then no error is raised if the file does not exist. If **RESULT_VARIABLE** is given, the variable `<var>` will be set to the full filename which has been included or **NOTFOUND** if it failed.
 - Examples:

```
include(TestBigEndian RESULT_VARIABLE mpath)
test_big_endian(isBigEndian)
message("Is target system big endian: ${isBigEndian}")
message("Path to the module: ${mpath}")
```

```
#include FetchContent to be able to download GoogleTest code.
include(FetchContent)

# Declare git repo and tag to fetch:
FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG v1.15.2
)

#Download or check that googletest is downloaded:
FetchContent_MakeAvailable(googletest)
```

- ❖ The CMake software suite includes the CTest tool which can be used to automate the testing phase. To use **ctest**, in the CMakeLists.txt we shall enable testing and add tests. Testing shall be enabled in the top level CMakeLists.txt file.
- ❖ To enable testing use: `enable_testing()` and to add a test use `add_test()`
- ❖ `add_test()` adds a new test called testName which runs the specified command with the given arguments
- ❖ Example:

```
enable_testing()
add_test(NAME test_led COMMAND test_led)
add_test(NAME test_ldr COMMAND test_ldr)
```
- ❖ Then you can run **ctest** in where the test executables are. E.g. **cd build && ctest**

```
add_test(NAME testName
COMMAND command [arg...]
[CONFIGURATIONS config1 [config2...]]
[WORKING_DIRECTORY dir]
)
```

Build Automation

❖ Some useful links

- [The GCC Manual](#)
- [UPEvent: GCC and Makefiles](#)
- [C Programming: Makefiles](#)
- [GNU make Manual](#)
- [A Shallow Dive into GNU Make](#)
- [GCC and Make](#)
- [Managing Projects with GNU Make](#)
- [The GNU Profiler \(gprof\) Manual](#)
- [How to use Linux GNU GCC Profiling Tool](#)
- [GPROF - Unix, Linux Command](#)
- [CMake Tutorial](#)
- [Mastering CMake](#)